UFPEL

LEITURA DE PAPER · NEUROEVOLUÇÃO

Evolução epigenética de redes convolucionais profundas

Hadjiivanov & Blair · UNSW Sydney · arXiv:2104.05411 (2021)

Andrew Borges Campos · Universidade Federal de Pelotas · Novembro de 2026

ROTEIRO

O que vamos ver

1. **Contexto**
   Neuroevolução e uma revisão rápida de CNNs

2. **O método**
   Cortex, crossover em camadas e kernels de formato livre

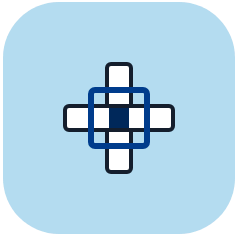
3. **Experimentos**
   Configuração, resultados e o papel da especiação

4. **Leitura crítica**
   O que o paper prova e o que fica em aberto

VISÃO GERAL

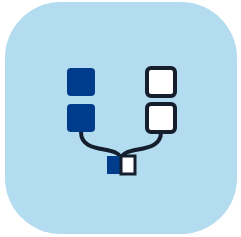
O paper em 30 segundos

Uma **população de CNNs** evolui a sua estrutura ao longo de gerações; os pesos são treinados com backprop e **herdados pelos filhos**. Três contribuições:



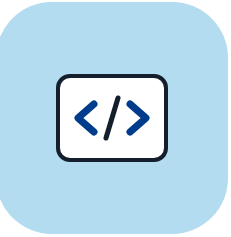
Kernels de formato livre

Cada kernel tem sua própria largura e altura (1×7, 3×3, 5×5...) **dentro da mesma camada**, com uma mutação que redimensiona kernels individuais.



Crossover em camadas

Camadas viram cromossomos e kernels viram genes: o filho herda **kernels inteiros, com pesos**, sem precisar "consertar" a rede.



PyCortex

Plataforma open source sobre **PyTorch**: cada indivíduo evoluído já é um modelo PyTorch válido, treinável com backprop.

CONTEXTO

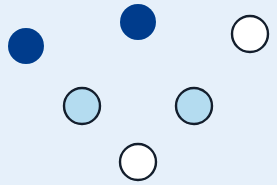
Neuroevolução: evoluir a rede em vez de projetá-la

- **Neuroevolução (NE)** = otimizar a estrutura e/ou os pesos de redes neurais com algoritmos evolutivos: população, mutação, crossover e seleção.
- Começou com redes pequenas (pole balancing, labirintos). **NEAT** (2002) popularizou evoluir topologias protegendo inovações com **especiação**.
- Tendência recente, que o paper segue: a evolução busca a **arquitetura**, o backprop ajusta os **pesos** — um otimizador híbrido.

EVOLUÇÃO · BUSCA GLOBAL

Estrutura

nº de camadas, nº de kernels, formato de cada kernel, stride



+

BACKPROP · AJUSTE LOCAL

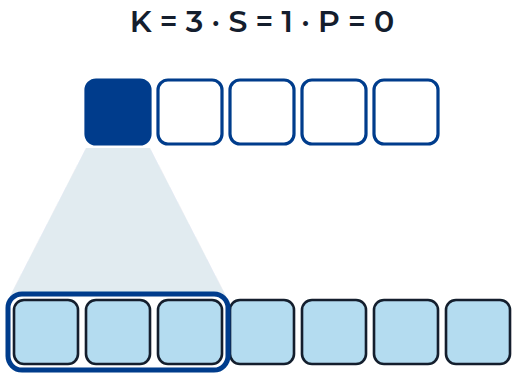
Pesos

gradiente descendente (Adadelta) sobre os pesos herdados

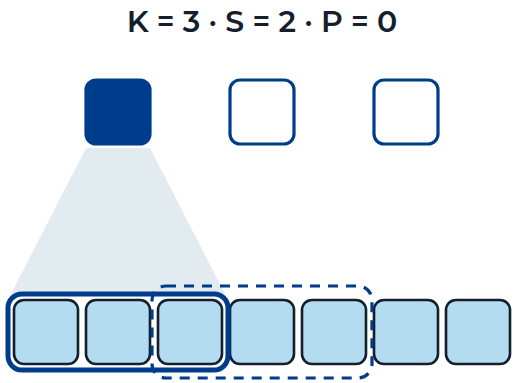


CONTEXTO

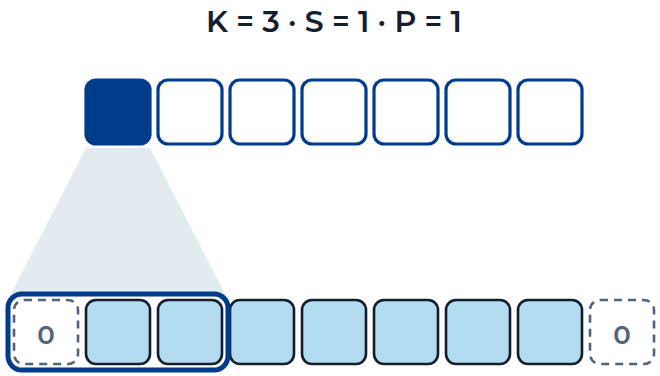
Revisão rápida: kernel, stride e padding



saída menor: 5 valores



stride 2: pula posições → 3



padding: saída = entrada (7)

COMO FUNCIONA

Cada kernel desliza sobre a entrada e produz um **mapa de resposta**. Camadas conv extraem features; camadas FC classificam.

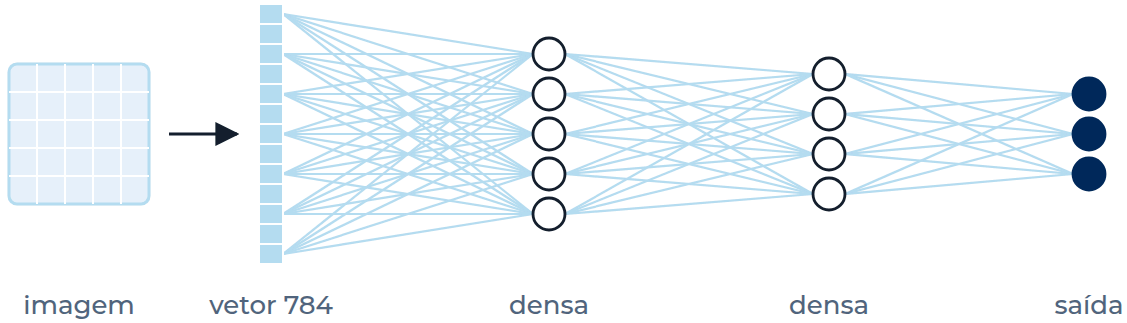
A CONVENÇÃO QUE O PAPER QUEBRA

Todos os kernels de uma camada têm o mesmo D×W×H: o tamanho do kernel vira **atributo da camada**.

CONTEXTO

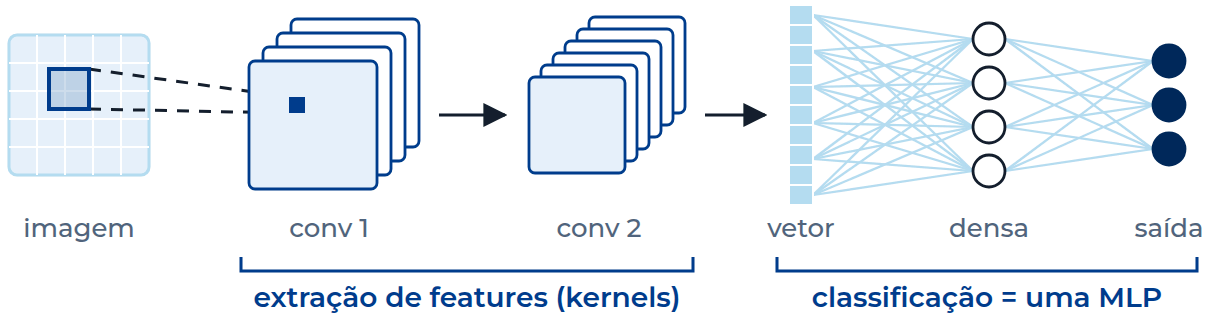
MLP × CNN: a arquitetura completa

MLP



Tudo é camada densa
cada neurônio vê a entrada inteira; a imagem vira vetor e a vizinhança entre pixels se perde

CNN



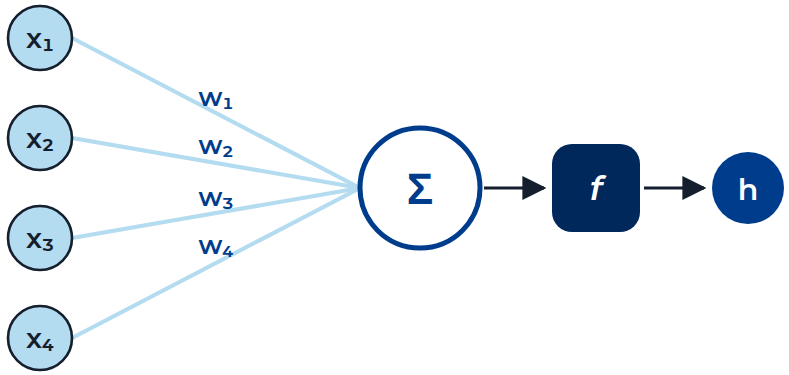
Kernels locais e compartilhados
poucos pesos e o mesmo detector em toda a imagem; cada kernel gera um mapa de resposta

Uma CNN = extrator de features + uma MLP no final. O paper evolui as duas partes, mas sem pooling: só camadas conv e FC.

CONTEXTO

O kernel é um neurônio que desliza

MLP: um neurônio



h = f(w · x + b) — vê a entrada inteira

CNN: um kernel

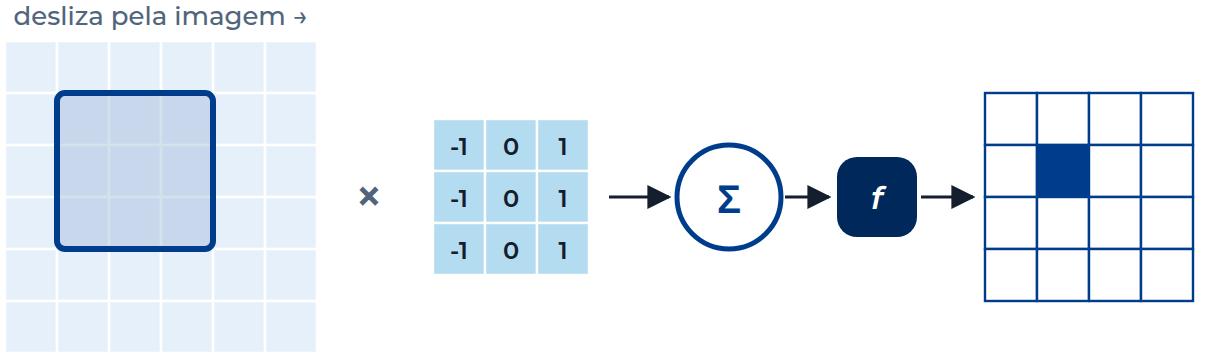


imagem · janela 3×3 | kernel = pesos | mapa de resposta

y[i,j] = f(kernel · janela[i,j] + b) — vê uma janela por vez

PESOS FIXOS NO FORWARD

O **mesmo kernel** é usado em todas as posições. Os pesos só mudam no backprop, entre um lote e outro.

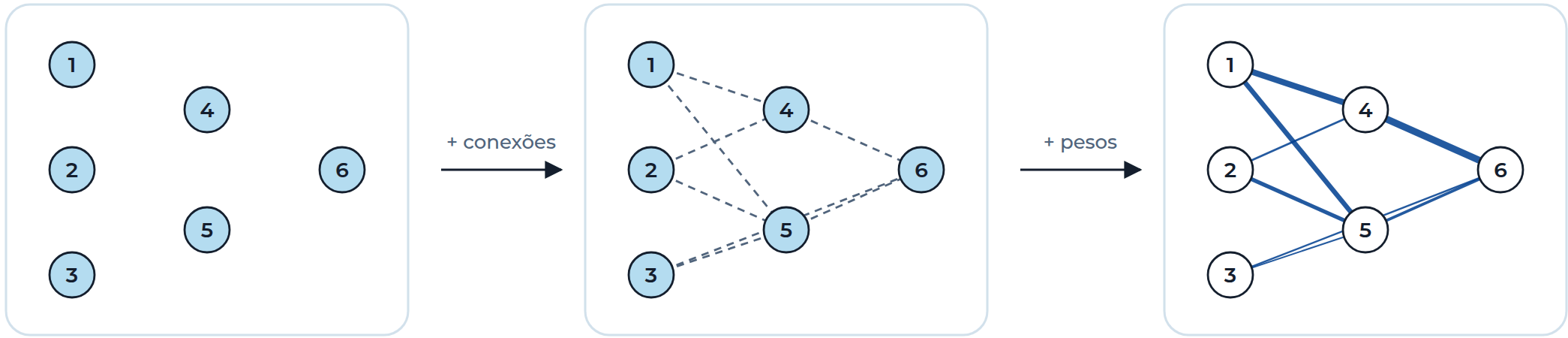
UMA RESPOSTA POR POSIÇÃO

Soma ponderada **janela × pesos** + bias, depois a ativação: um valor do mapa.

NO PAPER

Nó = kernel: adicionar um kernel é adicionar um neurônio; o crossover herda kernels inteiros.

O framework Cortex: genoma, genótipo e fenótipo



Genoma
esqueleto de nós ordenados

Genótipo
conexões, ainda sem pesos

Fenótipo
a rede real, com pesos

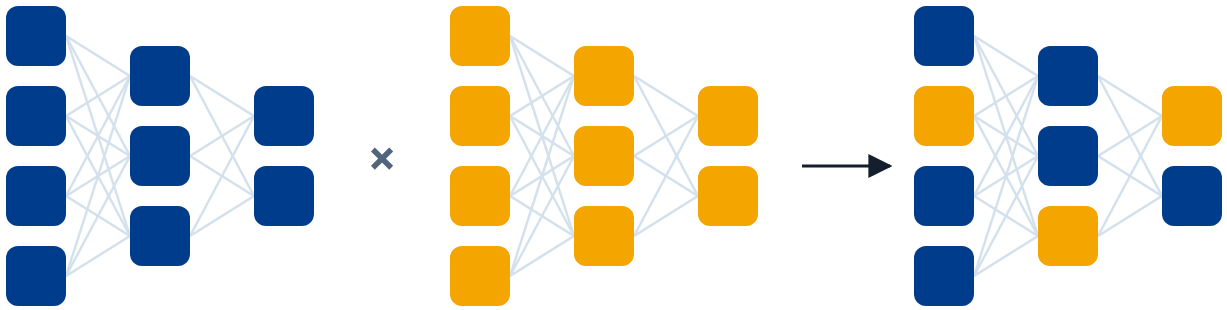
ESPECIAÇÃO

Redes com o **mesmo genoma** formam uma espécie e só competem entre si — mutações recentes ganham tempo para amadurecer.

POR QUE "EPIGENÉTICA"

Os pesos aprendidos durante a "vida" dos pais são **herdados sem alteração** pelos filhos (evolução lamarckiana).

Crossover em camadas: kernels como genes



Pai 1
fitness relativo 0,7

Pai 2
fitness relativo 0,4

Filho
genes dos dois pais

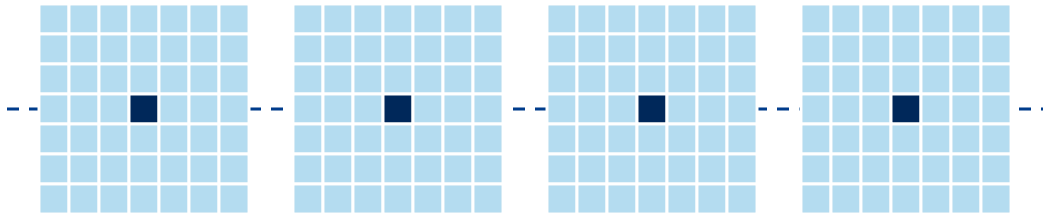
gene do pai 1 gene do pai 2

coluna = camada (cromossomo) · quadrado = kernel (gene)

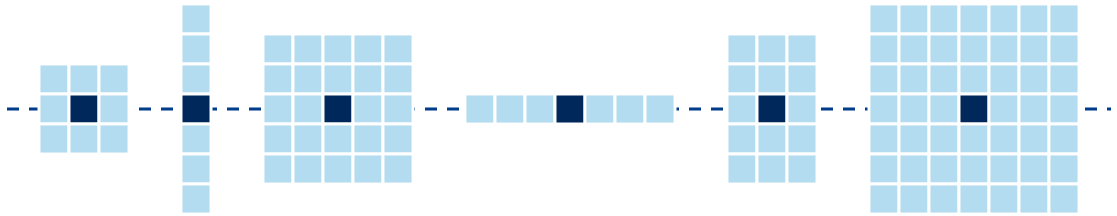
- **Gene indivisível** = um nó (kernel) com todas as suas conexões de entrada e pesos.
- Com especiação, os pais têm as mesmas camadas e tamanhos → o filho é **sempre funcional**, sem adicionar nem podar conexões.
- Cada gene vem de um pai com probabilidade proporcional ao seu **fitness relativo**.
- Sem especiação: a similaridade $s = O / (N_i + N_j - O)$ decide quem cruza; o excedente vem do pai maior.

A ideia central: kernels de formatos diferentes na mesma camada

Convencional: o formato é da camada

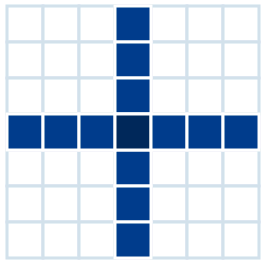


Proposto: o formato é de cada kernel



Como cabe num tensor só: W e H são sempre ímpares, então todos os kernels se alinham pelo **elemento central** (escuro) e são empilhados antes da convolução. Inspiração: o módulo Inception, que já mistura 1×1, 3×3 e 5×5.

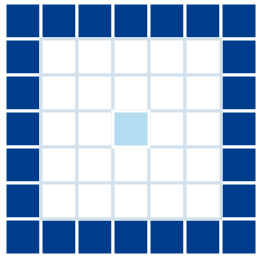
Por que misturar formatos ajudaria?



Detector de cruz

1×7 + 7×1

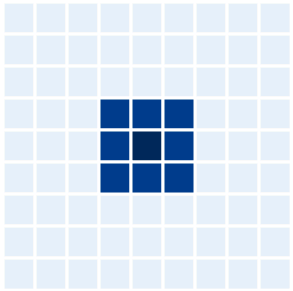
Kernels longos e finos detectam **bordas nítidas** sem "poluição" dos vizinhos; um par ortogonal detecta cruzamentos.



Detector de contorno

7×7 − 5×5

A diferença entre dois kernels concêntricos responde ao que **envolve o centro** — numa única camada.



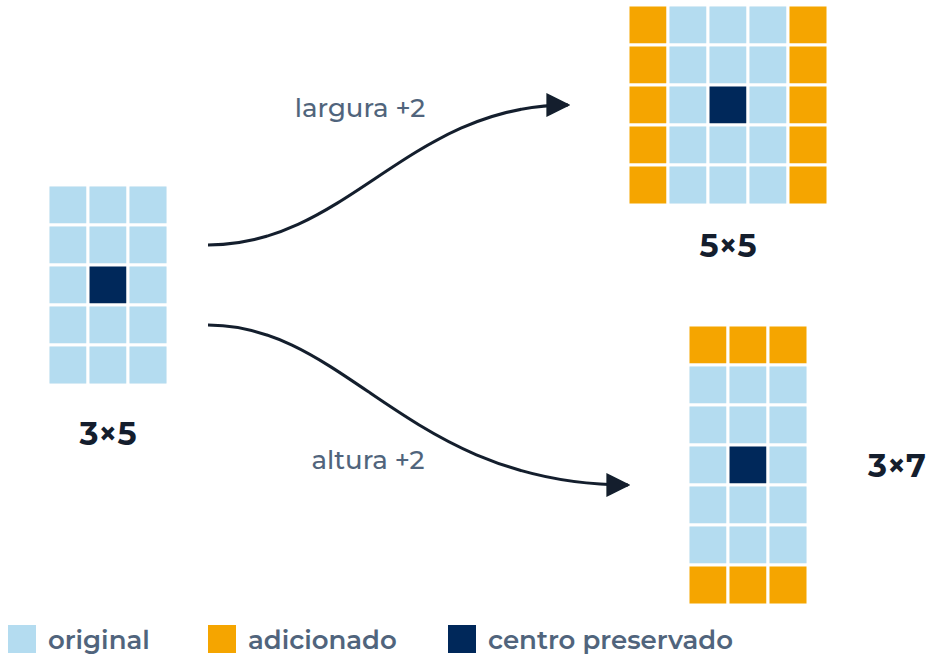
Contexto

9×9 junto de 3×3

Kernels grandes dão **contexto** aos detalhes finos dos pequenos, já disponível para a camada seguinte.

Argumento dos autores — o paper **não testa** essa hipótese isoladamente (a análise dos kernels ficou para trabalho futuro).

Como a estrutura muta



- **Kernel:** cresce ou encolhe W ou H em 2 — continua ímpar e o centro é preservado. Limites: 1×1 até metade da entrada.
- **Stride:** ±1 em W ou H. Rara, porque muda o tamanho de todas as camadas acima.
- **Padding:** não muta; vale ⌊maior kernel / 2⌋, então a saída mantém o tamanho da entrada.
- **Nós e camadas:** adicionar/remover kernels e camadas (conv ou FC).
- Probabilidade de cada mutação ∝ 1 / conexões afetadas. Formato inicial com $p \propto e^{-W \cdot H}$: **começa pequeno**.

Uma geração, passo a passo

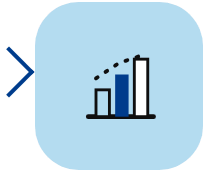
Etapa 01



Treino e avaliação

Backprop (Adadelta) em **10% aleatórios** do treino; fitness = acurácia no teste.

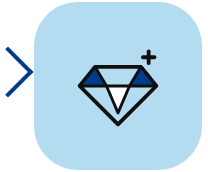
Etapa 02



Calibração

Fitness relativo $f_r = \sigma((f - \mu) / \sigma_f) \in (0, 1)$, usado como probabilidade.

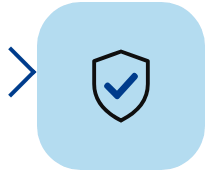
Etapa 03



Evolução

Mutações estruturais + crossover; pais sorteados ∝ fitness relativo, dentro da espécie.

Etapa 04



Culling

Acima do limite, remove redes com $p \propto \text{idade} / (f_r \cdot \text{complexidade})$. Filhos e campeões ficam.

Ciclo repetido por 100 gerações. Início com especiação: 8 espécies, a primeira mínima (só a camada de saída olhando a entrada) e as outras geradas por mutações aleatórias.

EXPERIMENTOS

Configuração dos experimentos

Datasets

4

MNIST e Fashion-MNIST (cinza) · SVHN e CIFAR-10 (RGB natural)

Execuções × gerações

10 × 100

por experimento, com e sem especiação

Do treino por época

10%

sorteados por rede: nenhuma rede vê o dataset inteiro

Redes na população

64 → 111

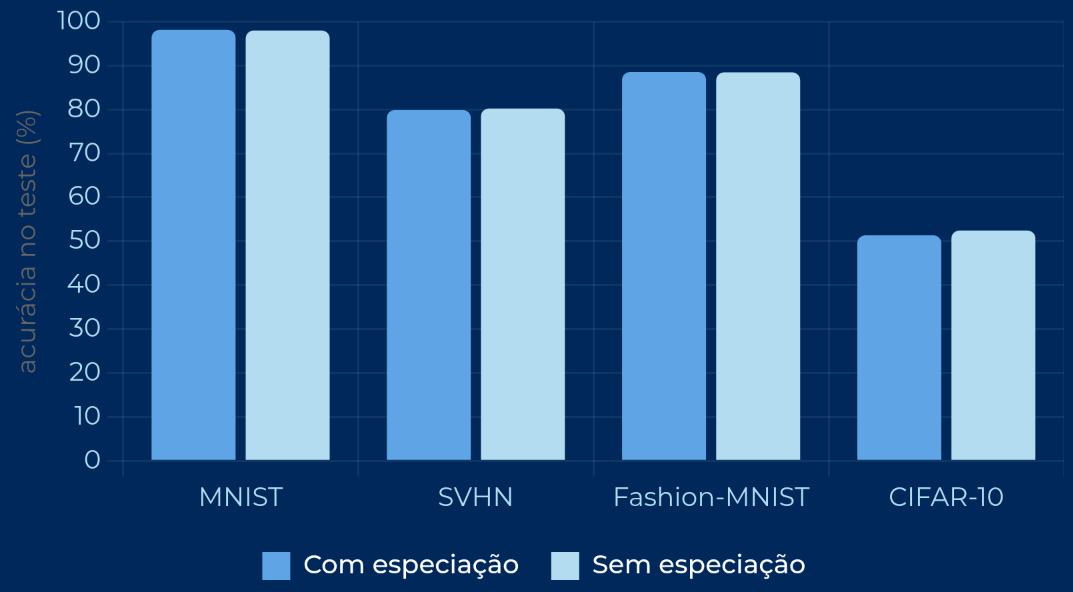
inicial → máximo, uma CPU por rede

Detalhes: só camadas conv e FC (sem pooling) · batch 128 · pesos iniciais N(0; 0,1) · 8 espécies iniciais, no máximo 16 · cada filho é avaliado no teste **antes de qualquer treino**.

EXPERIMENTOS · RESULTADOS

O efeito da especiação aparece nos filhos

Melhor rede da população



Praticamente empatado — em SVHN e CIFAR-10, sem especiação fica até à frente. Média de 10 execuções, geração 100.

Filhos recém-nascidos, antes do backprop



Com especiação, os filhos nascem 10 a 23 p.p. melhores. Média de 10 execuções, geração 100.

LEITURA CRÍTICA

O que o paper mostra — e o que falta

O ACHADO PRINCIPAL

Nenhuma rede viu o dataset inteiro, mas a população viu. Filhos acertam **~96% no MNIST sem treino** — só é possível se o crossover **transfere features aprendidas** pelos pais.

Especiação alinha os genes no crossover: filhos melhores (95,6% vs 72,5% no MNIST), mas redes que quase não crescem.

LIMITAÇÕES

- **Redes pequenas** (~10^5 parâmetros): CIFAR-10 fica em ~55%. O limite de 16 espécies travou mutações estruturais.
- Kernels de formato livre **sem ablação**: não se mede se ajudam de fato.
- Fitness = acurácia **no conjunto de teste**: a seleção "enxerga" o teste, o que infla os números.
- Só 10 execuções, **sem teste estatístico** — os próprios autores pedem mais experimentos.

UFPEL

Para aprofundar

Hadjiivanov, A.; Blair, A. *Epigenetic evolution of deep convolutional models.* arXiv, 2021 · arxiv.org/abs/2104.05411

Código: PyCortex · gitlab.com/cantordust/pycortex

Base: Stanley & Miikkulainen (2002), NEAT · Szegedy et al. (2015), Inception